哈爾濱工程大學

第 08 讲 / 全 17 讲

C 语言数组用法详解

一个名字，管住一整排数据 —— 从一维到二维，再到字符串

一维数组

一排连号的格子，下标从 0 开始

二维数组

成绩表矩阵：一行一个学生

字符数组

字符串就是字符排成的队

数组传参

交给函数的是这一排的起点

主讲教师：郎大鹏

本节目标

学完你应该能做到

哈爾濱工程大學

1. 说清为什么需要数组：用数组摆脱「30 个成绩起 30 个变量名」
2. 会定义一维数组，会用下标读写，记得下标从 0 开始
3. 理解越界的代价，养成「从 0 开始、小于 n 结束」的习惯
4. 会定义二维数组，会用双重 for 按行、按列处理成绩表
5. 理解字符数组与字符串的关系，记住结尾那个结束标记
6. 知道数组传给函数的是什么，为什么函数里改数组会带回来

01

一维数组

为什么需要数组 · 内存里的样子 · 定义与初始化 · 下标与遍历 · 越界的代价

一个名字管住一整排格子 —— 这是本课全部内容的基础

哈爾濱工程大学

为什么需要数组

从 30 个成绩说起

哈尔滨工程大学

要存全班 30 个成绩：一个变量只能存一个数，难道要起 30 个变量名？

不用数组：30 个变量

```
int s1, s2, s3, s4, s5;
int s6, s7, s8, s9, s10;
// 还要一直写到 s30
scanf("%d", &s1);
scanf("%d", &s2);
// 30 行输入，一行不能少
sum = s1 + s2 + s3 + s4;
sum = sum + s5 + s6 + s7;
sum = sum + s8 + s9 + s10;
// 手加到 s30，漏一个就错
```

用数组：一个名字

```
int s[30];
int i, sum = 0;
// 30 个格子，一个 s 全管住
for (i = 0; i < 30; i++)
    scanf("%d", &s[i]);
for (i = 0; i < 30; i++)
    sum = sum + s[i];
printf("%d", sum);
// 改成 300 人只改一个数字
```

数组在内存里长什么样

记住这张格子图

哈尔滨工程大学

```
int a[10] = {72, 85, 91, 64, 78, 66, 90, 55, 88, 73};
```

10 个 int 格子，一个挨一个

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
72	85	91	64	78	66	90	55	88	73

一排连号的格子

int a[10] 占下 10 个紧挨着的 int 格子，中间没有缝。

一个不落，挨着排

下标从 0 开始

排头叫 a[0]，排尾叫 a[9]。10 个格子编号是 0 到 9。

长度 10，下标 0 到 9

每个格子同类型

10 个格子全是 int，都能读能写，用法和普通变量一样。

int 数组全是 int

定义数组的三种写法

类型 + 名字 + 长度

哈尔滨工程大学

```
// 写法一：只定义，先不开值
int a[10];
a[0] = 72;       // 之后再逐个赋值
a[1] = 85;
// 写法二：定义时就全部初始化
int b[5] = {72, 85, 91, 64, 78};
// 写法三：只给前几个，后面补 0
int c[5] = {1, 2};
// c 里面其实是：1 2 0 0 0
// 方括号里的长度要写明确
printf("%d", b[2]); // 输出 91
// 只定义不赋值，格子里是随机数
```

写法一：只定义

先开出 10 个格子，里面是随机值，赋值后再用。

写法二：全初始化

大括号里按顺序给初值，依次填进前几个格子。

写法三：给一部分

初值给少了不要紧，剩下的格子自动补 0。

下标访问与遍历

下标从 0 开始

哈尔滨工程大学

```
int a[5] = {72, 85, 91, 64, 78};
int i;
a[0] = 100;                 //  写：改  0  号格子
printf("%d", a[0]);         //  读：打印  100
//  用  for  把每个格子走一遍
for (i = 0; i < 5; i++)
    printf("%d ", a[i]);
//  输出：  100 85 91 64 78
//  最后一个格子是  a[4]，不是  a[5]
printf("%d", a[5]);         //  错：越界
```

下标就是编号

a[i] 就是第 i 号格子，能读也能写，用起来和普通变量一样。

从 0 开始数

长度 5 只有 0 到 4 号，a[5] 已经是别人的地盘。

遍历靠 for 数

i 从 0 数到 n-1 ，循环体里用 a[i] 访问当前格子。

初始化的三个细节

补 0 · 清零 · 省长度

哈尔滨工程大学

初始化看着简单，真正容易记混的就是这三件事：补 0 、清零、省长度。

细节一：补 0

```
int a[5] = {1, 2};
// 剩下三个格子补 0
```

结果：1 2 0 0 0

初值个数比长度少时，没提到的格子一律填 0 ，不是随机值。

细节二：清零

```
int a[100] = {0};
// 只写一个 0，其余补 0
```

结果：100 个格子全是 0

想让整个数组从 0 开始，写这一个 0 就够了，不用写 100 遍。

细节三：省长度

```
int a[] = {1,2,3,4,5,6};
// 长度由初值个数定
```

结果：编译器数出长度 6

方括号里不写数字，编译器按你给的初值个数开格子。

注意分清两件事：只定义不初始化，格子里的值是随机的；初始化时没写到的格子，一律补 0 。

所以「先定义、再赋值」的时候，千万别忘了给每个用到的格子赋值。

越界的代价

C 不替你检查下标

哈尔滨工程大学

int a[5]; 只开了 5 个格子，下标 5 和更大的数都不属于这个数组。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	再往后
72	85	91	64	78	越界	越界	都不是你的

读越界 读出一个莫名其妙的值，等于翻了邻居的抽屉。

写越界 把别人的变量悄悄改掉，结果错得毫无规律。

运气差 碰了系统不让碰的区域，程序当场崩溃。

最常见的两种越界

```
// 错一：写成小于等于，多走一格
for (i = 0; i <= 5; i++)
    a[i] = 0;
// 错二：从 1 开始，漏了 0 多了 5
for (i = 1; i <= 5; i++)
```

正确写法只有一种： for (i = 0; i < n; i++)

02

二维数组

成绩表矩阵 · 定义与初始化 · 双重 for 遍历 · 按行按列求和 · 内存里也是一排

一维是一排格子，二维就是一张表 —— 一个下标变两个下标

哈爾濱工程大学

二维数组是什么

行 = 学生，列 = 科目

哈尔滨工程大学

一维是一排格子，二维就是一张表：横向看是一行，竖向看是一列。

行号 / 科目	第 0 列 语文	第 1 列 数学	第 2 列 英语	第 3 列 物理
a[0] 学生 1 第 0 行	**72**	**85**	**91**	**64**
a[1] 学生 2 第 1 行	**66**	**90**	**55**	**88**
a[2] 学生 3 第 2 行	**78**	**73**	**82**	**69**

写法是 a[行][列] ：先说第几行，再说第几列。 int a[3][4] 就是 3 行 4 列，一共 12 个格子。

二维数组的定义与初始化


```
// 定义：3 行 4 列，一共 12 个格子
int a[3][4];
// 分行初始化：一层大括号写一行
int b[3][4] = {
    {72, 85, 91, 64},
    {66, 90, 55, 88},
    {78, 73, 82, 69}
};
// 也可以写成一长串，按顺序填
int c[3][4] = {72,85,91,64,
               66,90,55,88,
               78,73,82,69};
```

两个方括号

int a[3][4] ：前面是行数，后面是列数，访问写成 a[行][列] 。

分行加大括号

每行再用一层大括号包起来，行列一目了然，推荐这样写。

第一维可以省

int a[][4] ：行数交给编译器数，列数一定不能省。

双重 for 遍历矩阵

外层走行，内层走列

哈尔滨工程大学

```
int a[3][4] = {
    {72, 85, 91, 64},
    {66, 90, 55, 88},
    {78, 73, 82, 69}
};
int i, j;
for (i = 0; i < 3; i++)
//  外层循环：一次走一行
{
    for (j = 0; j < 4; j++)
    //  内层循环：走完这一行
        printf("%4d", a[i][j]);
    //  这一行结束再输出一个换行
}
```

走格子的顺序

外层 i 定住一行，内层 j 把这一行走完。

1	2	3	4
5	6	7	8
9	10	11	12

打印出来的样子

```
72   85   91   64
66   90   55   88
78   73   82   69
```

一行一个学生，一列一门课。

按行求和 / 按列求和

换哪个循环放外面

同一张表，想算「每个人的总分」还是「每门课的平均」，只要换一下谁在外面。

按行：每个学生的总分

```
int sum;
for (i = 0; i < 3; i++)
//  外层：一个一个学生来
{
    sum = 0;
    //  换人了，累加器要清零
    for (j = 0; j < 4; j++)
        sum = sum + a[i][j];
    printf("%d", sum);
    //  输出：312 299 302
}
```

按列：每门课的平均分

```
int sum;
for (j = 0; j < 4; j++)
//  外层：一门一门课来
{
    sum = 0;
    //  换科目了，累加器要清零
    for (i = 0; i < 3; i++)
        sum = sum + a[i][j];
    printf("%d", sum / 3);
    //  输出：72 82 76 73
}
```

二维数组在内存也是连续的

行和列只是看法的不同

哈尔滨工程大学

我们把它画成一张表，但内存里根本没有表格线，只有一长排挨着的格子。

a[0][0]	a[0][1]	a[0][2]	a[0][3]
72	85	91	64
a[1][0]	a[1][1]	a[1][2]	a[1][3]
66	90	55	88
a[2][0]	a[2][1]	a[2][2]	a[2][3]
78	73	82	69

> 内存里没有行列，只有一长排格子：第 0 行的 4 个走完，紧接着第 1 行。

72	85	91	64	66	90	55	88	78	73	82	69

遍历顺序的性能直觉

了解即可，不必深究

哈尔滨工程大学

两层循环谁放外面？结果一样、跑的次数也一样，差别只在「走的路线」。

顺着走：外层行，内层列

```
// 外层走行，内层走列
for (i = 0; i < 3; i++)
    for (j = 0; j < 4; j++)
        sum = sum + a[i][j];
// 走法：00 01 02 03 10 11 ...
```

跳着走：外层列，内层行

```
// 外层走列，内层走行
for (j = 0; j < 4; j++)
    for (i = 0; i < 3; i++)
        sum = sum + a[i][j];
// 走法：00 10 20 01 11 21 ...
```

数组按行存放：顺着行走，访问的是紧挨着的格子；跳着走，每隔一个就要跨过一大段。先养成「外层行、内层列」的习惯。

03

字符数组与数组传参

字符数组与字符串 · 结束标记 · %s 输入输出 · 逐字符遍历 · 传首地址 · 改了会带回来

字符排起来就是字符串，数组进函数传的是起点 —— 这里埋着指针课的伏笔

哈爾濱工程大學

字符数组与字符串

每个格子一个字符

哈尔滨工程大学

char s[8] = "hello"; —— 这一句在内存里占了 8 个格子，一个格子放一个字符。

h	e	l	l	o	\0	空着	空着
s[0]	s[1]	s[2]	s[3]	s[4]	s[5] 句号	s[6]	s[7]

就是一排 char

char s[8]; 开出 8 个格子，每个放一个字符，写法和 int 数组一样。

要多留一个位置

5 个字母的 hello 要用 6 个格子，多出来的那个专门放结束标记。

可以直接写串

char s[8] = "hello"; 比逐个写字符方便，结束标记自动补上。

字符数组的重点只有两条：格子里放的是字符；末尾要留一个位置放结束标记。

'\0' 是字符串的句号

本课最重要的一个字符

哈尔滨工程大学

字符串末尾必须有一个 '\0'，它是「到此为止」的句号。没有它，程序就不知道该在哪儿停下。

有句号：读到 s[5] 就停下，老老实实输出 hello

h	e	l	l	o	\0	没用上	没用上
s[0]	s[1]	s[2]	s[3]	s[4]	s[5] 停	s[6]	s[7]

没有句号：程序不知道停在哪，一直往后读，把别人的格子也读出来

h	e	l	l	o	?	?	乱码
s[0]	s[1]	s[2]	s[3]	s[4] 满了	接着读	接着读	停不下来

结束标记编码为 0，不是数字 0 也不是字符 0；它不显示，只标记到此结束。

多留一格：存 5 个字符至少开 char s[6]。

用 %s 输入输出字符串

整体读写，一次到位

哈尔滨工程大学

```
char name[20];
// 读：格式符用 %s，写数组名
scanf("%s", name);
// 写：同样用 %s，一次输出整串
printf("%s", name);
// 单个字符也能单独取出来
printf("%c", name[0]);
// 输入：Zhang San
// name 只拿到 Zhang，空格后停
// 20 个格子最多存 19 个字符
// 还要留一个给结束标记
```

%s 配数组名

scanf 里直接写数组名 name，不用再加别的符号。

遇到空格就停

读了 Zhang San 只拿到 Zhang，空格后面那半截还留在输入里。

输出打到句号停

用 %s 从第一个字符开始打印，一直打到结束标记才收工。

逐字符遍历：手写求长度

任务 不借助任何现成函数，自己一个一个数出字符串里有几个字符。

```
char s[100];
int i, len = 0;
scanf("%s", s);
// 写法一：用 for 数（本课主线）
for (i = 0; s[i] != '\0'; i++)
    len = len + 1;
printf("%d", len);
// 输入 hello，输出 5
// 写法二：用 while 也完全可以
i = 0;
while (s[i] != '\0')
    i = i + 1;
```

条件不再是 i < n

字符串有多长事先不知道，条件只能写成「还没走到结束标记」。这正是字符数组和普通数组最大的区别。

用 while 也完全可以

前面学过的 while 一样能写，两种写法完全等价。以后遇到的现成求长度工具，内部也是这样一步步数出来的。

数组作参数传的是首地址

指针课的伏笔

哈尔滨工程大学

把数组交给函数时，C 不会复制一整排格子，而是告诉函数「这排格子从哪儿开始」。

```
int sum(int a[], int n)
{
    int s = 0, i;
    for (i = 0; i < n; i++)
        s = s + a[i];
    return s;
}
// 调用：只写数组名，后面不跟方括号
int b[5] = {1, 2, 3, 4, 5};
printf("%d", sum(b, 5));
// 输出 15
```

同一排格子，两个名字

main 里叫 b，函数里叫 a，指的是同一段内存。

1	2	3	4	5

b[0] 就是 a[0]，改任何一个，另一个也跟着变。

形参写成 int a[]，写 a[10] 也行

长度看不出来，必须把 n 一起传进去

函数里改数组会带回来

和值传递正相反

学函数时说过：普通变量传进去的是副本，函数里改不动。数组正好相反。

普通变量：值传递

```
void add(int x)
{
    x = x + 1;
    //  改的是副本，原件没动
}
add(a);
printf("%d", a);  //  还是 10
```

数组：传的是起点

```
void add(int b[], int i)
{
    b[i] = b[i] + 1;
    //  改的就是原来那一格
}
add(a, 0);
printf("%d", a[0]); //  变成 11
```

传进去的是这排格子的起点，所以函数里写 b[i] 改的就是 main 里那一格。为什么会有这种差别？指针课揭晓。

例 1 · 求平均分

读入 · 存数组 · 再求平均

例 1 先读人数 n ，再读 n 个成绩存进数组，最后求平均分（保留两位小数）。

```
int main(void)
{
    int s[50], n, i;
    double sum = 0;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &s[i]);
    for (i = 0; i < n; i++)
        sum = sum + s[i];
    printf("%.2f", sum / n);
    return 0;
}
```

运行结果

输入： 5

72 85 91 64 78

输出： 78.00

要点

成绩先全部读进数组，再单独跑一个循环累加。求和变量用 double ，除法才不会丢掉小数。

数组开 50 个格子，实际只用了前 n 个，完全没问题。

例 2 · 高于平均分的人数

例 2 接例 1 ，统计有多少人的成绩高于全班平均分。

```
int s[50], n, i, cnt = 0;
double sum = 0, avg;
scanf("%d", &n);
for (i = 0; i < n; i++)
    scanf("%d", &s[i]);
//  第一遍：算出平均分
for (i = 0; i < n; i++)
    sum = sum + s[i];
avg = sum / n;
//  第二遍：一个一个数
for (i = 0; i < n; i++)
    if (s[i] > avg) cnt = cnt + 1;
printf("%d", cnt);
```

运行结果

输入： 5
72 85 91 64 78
平均分 78.00
输出： 2

为什么要两遍

第一遍结束才拿得到平均分，第二遍才有东西可比。想一遍搞定是不行的：走在前面的人还不知道平均是多少。

例 3 · 最大值与它的下标

例 3 找出最高分是多少，并报出它是第几个（下标从 0 算起）。

```
int s[50], n, i, max, k;
scanf("%d", &n);
for (i = 0; i < n; i++)
    scanf("%d", &s[i]);
max = s[0];
// 擂主先上台：第 0 个
k = 0;
// 记住擂主是几号
for (i = 1; i < n; i++)
// 从 1 号开始一个个挑战
    if (s[i] > max)
    { max = s[i]; k = i; }
printf("%d %d", max, k);
```

运行结果

输入： 5

72 85 91 64 78

输出： 91 2

擂台模型进数组

max 记住「谁最强」，k 记住「他是几号」。两个变量一起更新，缺一个就答不出下标。

i 从 1 开始就够了，因为 0 号已经站在台上。

例 4 · 二维数组按列求平均

例 4 3 个学生 4 门课，逐门课算出平均分。

```
int a[3][4] = {72,85,91,64,
               66,90,55,88,
               78,73,82,69};
int i, j, sum;
//  外层走列：一门一门课来
for (j = 0; j < 4; j++)
{
    sum = 0;
    //  内层走行：这门课的三个人
    for (i = 0; i < 3; i++)
        sum = sum + a[i][j];
    printf("%d", sum / 3);
}
```

运行结果

语文 72　　数学 82
英语 76　　物理 73
输出： 72 82 76 73

要点

把列号 j 定住不动，让 i 从 0 走到 2，这一列就被竖着走完了。

sum 必须放在外层循环里清零，放在外面就会把上一门课的分数一起算进来。

例 5 · 字符串逆序

手写首尾交换

哈爾濱工程大學

例 5 把一个字符串倒过来，不用任何现成函数。

```
char s[100], t;
int i, len = 0, j;
scanf("%s", s);
// 第一步：先数出长度
for (i = 0; s[i] != '\0'; i++)
    len = len + 1;
// 第二步：只交换前一半
for (i = 0; i < len / 2; i++)
{
    j = len - 1 - i;
    // 三步交换，借 t 中转
    t = s[i]; s[i] = s[j]; s[j] = t;
}
printf("%s", s);
```

运行结果

输入： hello

输出： olleh

要点

只交换前一半就够了；要是走完全程，每对都会换两次，等于换回原样。

交换必须借一个临时变量 t 中转，直接互相赋值会把值冲掉。

结束标记一直待在末尾，不用动它。

常见错误盘点

这六条几乎人人都踩过

哈尔滨工程大学

- ✗ 下标从 1 开始： for (i = 1; i <= n; i++) 漏掉 a[0] ，还多碰了 a[n] 。
- ✗ 越界读写：数组长度 10 却写 a[10] ， C 不检查，错了也不吭声。
- ✗ 忘了给结束标记留位置： 5 个格子的数组装不下 hello ，输出会带出乱码。
- ✗ 数组大小用变量： int a[n]; 在 C89 里不允许，长度必须是常量。
- ✗ 传参后想用 sizeof 求长度：求不出来，长度一定要当成参数一起传。
- ✗ 初始化忘了写大括号：写成 int a[5] = 1, 2, 3; 编译直接报错。

小结

数组是一排连号的格子，下标从 0 开始；二维数组是按行存放的表；字符数组末尾要留一个结束标记；数组进函数传的是这一排的起点。

悬念：数组名到底是什么？

是地址！为什么传进函数就能改原数组，指针课揭晓。

下一专题：排序算法

把一排数据按大小整理好，数组就是它的舞台。

哈爾濱工程大學